códigofacilito

Bootcamp · Módulo 4: Integración App + DB

Realtime con Supabase

Clase 3 de 6 · Supabase + Next.js



Rodrigo Méndez
rodrigomendez.dev

códigofacilito

📋 Tabla de contenido

/01 El problema y la meta

/02 Suscribirse a los cambios

/03 Realtime en el navegador

/04 Realtime, RLS y Presence

/05 Para cerrar la clase



códigofacilito

/01

El problema y la meta

Qué vamos a construir hoy, y por qué.

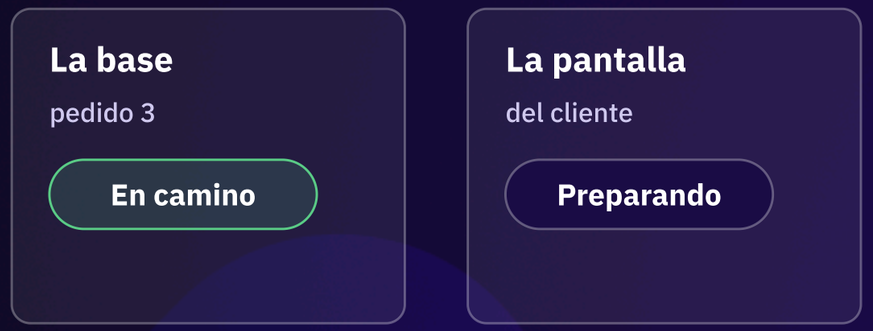


códigofacilito

EL PROBLEMA

La tienda marca un pedido "En camino". ¿Cómo se entera la pantalla del cliente sin que recargue?

HOY: LA PANTALLA SE QUEDA VIEJA



Solo cambia si el cliente recarga

LA SOLUCIÓN: LA BASE AVISA

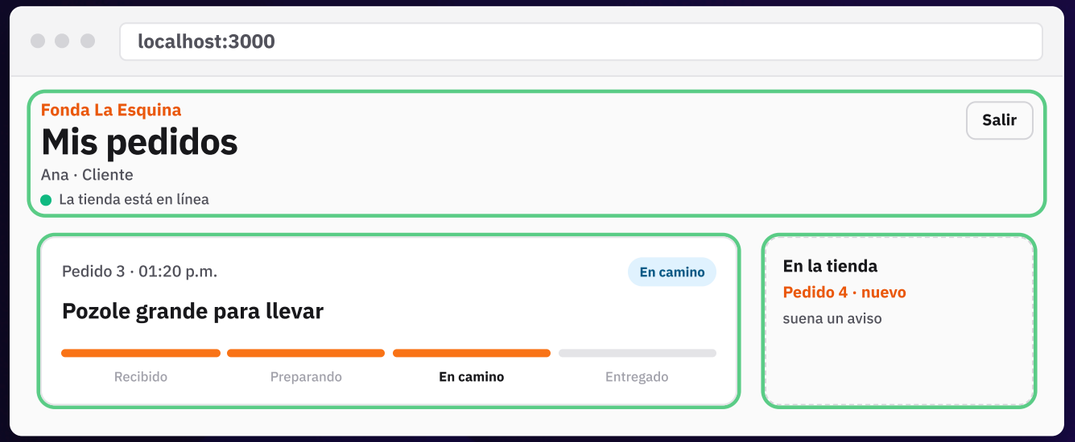


Sin recargar y sin preguntar a cada rato

códigofacilito

LA META DE HOY

La pantalla se actualiza sola



1. **La barra avanza sola**
   Cuando la tienda cambia el estado, sin recargar.
2. **Los pedidos nuevos llegan solos**
   A la tienda, con un sonido.
3. **Sabes si la tienda está conectada**
   Un indicador de "en línea".

RASTRO, SIGUE TU PEDIDO · UNA SOLA APP QUE CRECE CLASE A CLASE

CLASE 1	CLASE 2	CLASE 3	CLASE 4	CLASE 5	CLASE 6
		Hoy			
Base de datos	Auth y RLS	Realtime	Edge Functions	Storage	Eventos y Broadcast

códigofacilito

/02 Suscribirse a los cambios

Qué es Realtime y cómo llega un cambio.



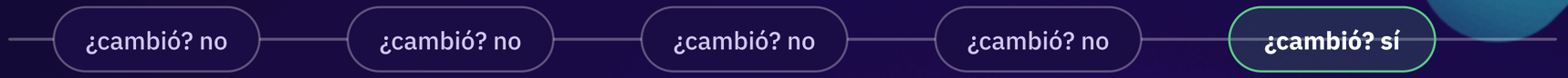
códigofacilito

DOS FORMAS DE ENTERARSE

Preguntar a cada rato o suscribirse

Preguntar cada 5 s

polling



Muchas peticiones para nada, y el cambio llega tarde.

Suscribirse

Realtime, por WebSocket

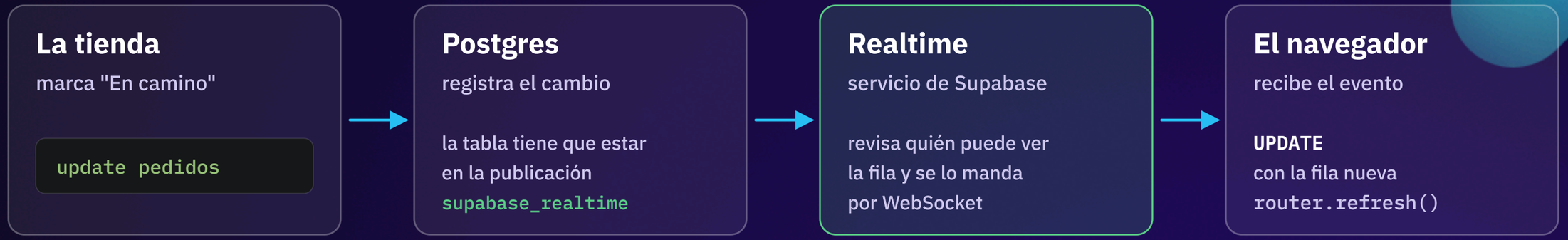


El servidor manda el cambio en cuanto ocurre.

códigofacilito

POSTGRES CHANGES

Cómo llega un cambio al navegador



Eventos: INSERT, UPDATE y DELETE. Te suscribes a los que te interesan, por tabla.

códigofacilito

EL ERROR DEL DÍA

No llega nada y no hay error

1

Te suscribes

El código es correcto y la conexión se abre.

2

No llega nada

La tabla no está en la publicación de Realtime.

3

Sin error

El servidor lo avisa en un mensaje de sistema.

4

Escucha "system"

Imprímelo en la consola: ahí dice qué falta.

> Si no llegan cambios, revisa que la tabla esté en `supabase_realtime` y escucha los mensajes de sistema del canal.

códigofacilito

/03

Realtime en el navegador

Un Client Component que se suscribe y se limpia.



códigofacilito

`"use client"`

La suscripción vive en el navegador

La página · Server Component

corre en el servidor: valida la sesión, consulta los pedidos y manda el HTML

`<EnVivo /> · Client Component`

corre en el navegador: se suscribe y no pinta nada
(regresa null)

→

`router.refresh()`

el servidor vuelve a renderizar la página
con los datos nuevos

Usa el cliente de Supabase del navegador: con la URL y la llave publishable, que son públicas.

códigofacilito

`useEffect`

Abrir y cerrar la suscripción

1

Monta

El componente aparece en la página.

2

Se suscribe

El efecto abre el canal y escucha los cambios.

3

Recibe avisos

Con cada evento, `router.refresh()`.

4

Se limpia

Al salir, `removeChannel` cierra el canal.

> Cada suscripción abierta es una conexión: si no la cierras en la limpieza del efecto, se acumulan.

códigofacilito

/04 Realtime, RLS y Presence

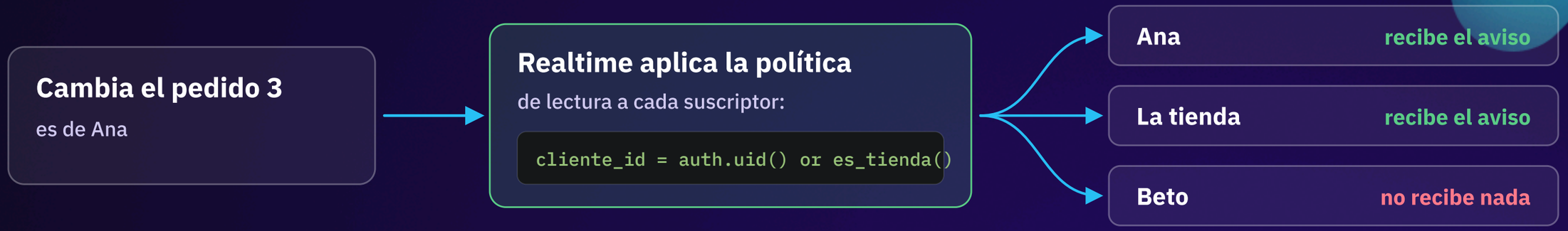
Quién recibe cada cambio y quién está conectado.



códigofacilito

REALTIME Y RLS

RLS también filtra lo que llega en vivo



Para saber quién es cada suscriptor, el navegador manda el token de la sesión antes de suscribirse.

`await supabase.realtime.setAuth()` Sin esta línea la suscripción entra como **anon**: ninguna política le deja ver filas y no llega nada, sin error.

códigofacilito

PRESENCE

Quién está conectado ahora

LA TIENDA

Se anota en el canal

Al conectarse, `track()` la agrega a la lista de presentes del canal. Al cerrar la ventana, sale sola.

LOS CLIENTES

Escuchan la lista

Con cada evento `sync` revisan quién está: si hay alguien anotado, la tienda está en línea.

> Presence no toca la base: es un estado compartido que dura lo que dura la conexión.

códigofacilito

/05

Para cerrar la clase

Lo que te llevas, la tarea y lo que sigue.



códigofacilito

LO QUE TE LLEVAS HOY

Tres ideas

1

Suscribirse en lugar de preguntar

Una conexión abierta y el servidor manda cada cambio al momento.

2

La suscripción vive en un Client Component

Se abre en `useEffect` y se cierra en su limpieza.

3

RLS también filtra lo que llega en vivo

Por eso el navegador manda su sesión con `setAuth()`.

códigofacilito

TAREA · PARA LA CLASE 4

Construye tú solo lo que viste hoy

CÓMO SÉ QUE YA QUEDÓ

- [ ] La tabla de pedidos está en la publicación de Realtime.
- [ ] La barra del cliente avanza sola cuando la tienda cambia el estado.
- [ ] A la tienda le suena un aviso con cada pedido nuevo.
- [ ] Un cliente no recibe los cambios de los pedidos de otro.

¿NO TE SALIÓ?

No pasa nada

Toma la base de la clase 4: ya trae todo esto hecho. Compárala con lo tuyo y encuentra qué te faltó.

códigofacilito

Recursos para profundizar

- Cambios de la base en vivo (Postgres Changes): supabase.com/docs/guides/realtime/postgres-changes
- Presence: supabase.com/docs/guides/realtime/presence
- `"use client"` en Next.js: nextjs.org/docs/app/api-reference/directives/use-client
- `useEffect` y su limpieza: react.dev/reference/react/useEffect



códigofacilito

LO QUE SIGUE

¿Quién le avisa al cliente por correo?

EN EL NAVEGADOR

No se puede

Mandar un correo necesita una llave secreta. En el navegador, cualquiera la copia y manda correos a tu nombre.

EN SUPABASE

Una Edge Function

Código que corre junto a la base, con la llave guardada como secreto.

Hoy la pantalla se actualiza sola. En la clase 4 el aviso sale por correo, sin que nadie apriete nada.